

A Comparative Analysis of kNN, Support Vector Machines, Decision Trees, and Neural Networks on the Adult Income and Wine Quality Datasets

Saumya Rada

I. EDA ON DATASETS

Exploratory data analysis (EDA) of the Adult income dataset shows that the dataset consists of 45,222 data points with 15 features. These are a mix of categorical variables (workclass, occupation, marital-status etc.) and continuous variables (age, capital-gain/capital-loss, hours-per-week, etc.). Some strong financial indicators like capital-gain and capital-loss, as in Figure 1, show an extreme right skew and sparsity with around 75% of the reported data being 0. Due to this sparsity, the model must often rely on other less reliable features such as age and education to predict income class. Many categorical variables have imbalanced frequencies, examples being white in race (86%), male in sex (2:1), and United States in native-country. Additionally, several categorical features contain many low-frequency categories, resulting in sparse high-dimensional representations after encoding. The target variable (income) also has a strong class imbalance with approximately a 3:1 ratio between $\leq 50K$ vs $> 50K$ income. Due to this, models can make inaccurate predictions, often predicting an income $\leq 50K$ unless there is strong evidence proving otherwise. Therefore, accuracy will be an insufficient performance metric. Income patterns appear to arise from conditional interactions between features such as education, occupation, and age rather than linear relationships. Given these factors, pruned decision trees and SVMs are expected to perform the best as they can capture conditional structure and are robust to noise. In contrast, kNN is more sensitive to high-dimensional features and Neural Networks are most sensitive to class imbalance.

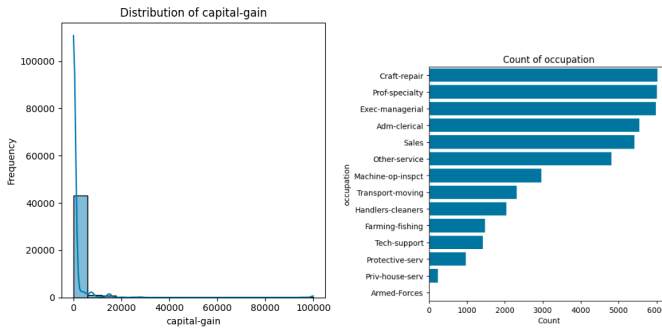


Fig. 1: Distribution of the capital-gain and occupation features in the Adult dataset.

EDA on the Wine Quality dataset shows us that the dataset has 6497 wines with 11 features. The class and

quality features are identical and one of them will need to be dropped before training to avoid leakage. There are no missing values. Aside from type, all of the features are continuous variables. Several of the features are right-skewed. Some examples shown through diagrams are residual_sugar, free_sulfur_dioxide, and total_sulfur_dioxide. The features free_sulfur_dioxide and total_sulfur_dioxide are strongly correlated as seen in the heat map in Figure 2. Alcohol is strongly negatively correlated with density, and there are moderate associations with quality as higher alcohol and sulphates tend to align with better quality. Higher volatile_acidity and type (1 - white wine, 0 - red wine) tend to align with lower quality. Multicollinearity between other features in the dataset is present but not extreme beyond these. Most feature-quality correlations are moderate between 0.3-0.4 and highly collinear groups are only weakly correlated with quality. There seems to be a large variance due to human perception, noise and unobserved variables. The class balance of the target variable (wine) is skewed towards mid-scored (3-5). Due to this, accuracy is less informative and F1 score will be more useful in assessing performance of the models. Overall, the features in the dataset provide little information to predict the target variable which may lead to the models not having very high performances.

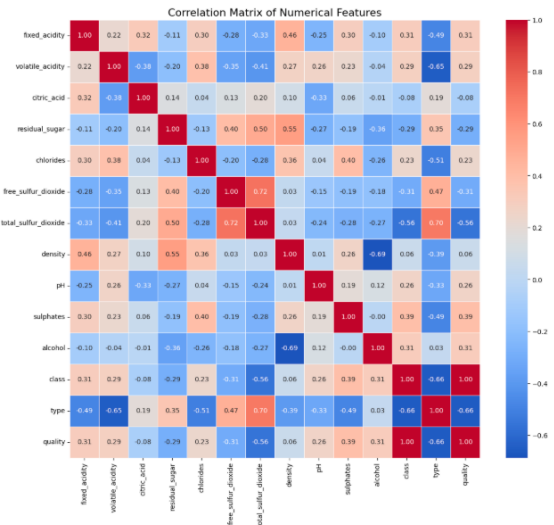


Fig. 2: Correlation matrix for Wine Quality dataset.

II. HYPOTHESES

On the Adult Income dataset, I hypothesize pruned Decision Trees $\geq$ SVM $\geq$ Neural Networks $\geq$ kNN in terms of F1-score. EDA shows us a mix of categorical and continuous features with high-dimensional categorical variables, weak correlations between features and income, heavily skewed numeric variables (e.g., capital-gain and capital-loss), and class imbalance with higher frequency of the $\leq 50K$ class. Pruned Decision Trees are expected to perform best because they naturally model non-linear interactions, are better suited for categorical variables and control variance through regularization such as pruning and depth/leaf constraints. SVMs use margin maximization and can effectively represent the high-dimensional encoded space but are sensitive to skew and require careful tuning. Neural Networks have high capacity and can learn incorrect patterns in the noise present in tabular, structured data, therefore increasing variance which is further increased by class imbalance. kNN might perform the worst due to sensitivity to local data and distance distortion in high dimensions spaces.

On the Wine Quality dataset I hypothesize, Neural Networks $\geq$ SVM $\geq$ Decision Trees $\geq$ kNN in terms of F1 score. EDA shows us fully continuous chemical features with skewed distributions, few linear correlations, limited sample size, and noise in the quality labels. Neural Networks are expected to perform best because they can model complex and non-linear interactions among chemical attributes. They have high capacity which allows them to do this. SVMs with RBF kernels perform well in continuous feature spaces and can be tuned with margin-based regularization to control variance. However, they have lower capacity than NNs. Decision Trees can have high variance with continuous variables and noisy targets, even after pruning. kNN should perform worst due to sensitivity to local data, distance distortion from skewed features, and noise amplification as they are dependent on the average of neighbors.

III. METHODS

For the Adult income dataset, the target variable was set as the income class for adults. The income class consists of two values, $\leq 50k$ and $> 50K$. This is a binary classification task. There was a heavy class imbalance of around 3:1 favouring the $\leq 50k$ class. Models trained on imbalanced datasets tend to favor majority-class predictions, making accuracy an unreliable metric compared to balanced measures such as F1-score (He Garcia, 2009). For example, a model that only predicts $\leq 50k$ will still be 75% accurate. Therefore, F1-score and AUC-ROC are going to be used to measure the performance of the model.

For the Wine Quality dataset, the target variable was the quality feature. This is a multi-class classification problem as there are 8 classes of wine types. The class balance of the quality variable is greatly skewed towards wines with scores 3-5. Similar to the adult dataset, accuracy is a misleading measure in this dataset. The macro F1 is used instead of the weighted F1 score as it weighs each class equally and shows lower recall in low-frequency classes (1,2,7,8). Additionally,

there was a leaky variable named class with identical information to the quality variable which was dropped from the training set to avoid inaccurate predictions.

Both the datasets were split three ways (70% training, 15% validation, and 15% testing) with stratification to preserve the class distribution. To ensure reproducibility, a fixed seed (=42) was used across all stochastic elements in the process. Hyperparameter tuning was performed on the training and validation sets, with the final held-out test set used only once to report the final performance of the four models.

To train the models, preprocessing, model-specific complexity tuning, and validation/test curves were implemented. ColumnTransformer was used for preprocessing that applied one-hot encoding to categorical variables and standardization to numerical features, ensuring all inputs have a similar scale. This helps distance-sensitive models (SVM, kNN, and NN).

Decision Tree optimization was done using the `cost_complexity_pruning_path` method to calculate the effective α values for the specific training set. 50 α values were sampled along this path and models were trained on the validation set to determine the best value that maximized the validation Macro F1-score. For SVMs, both a linear and RBF kernel were trained. First, the regularization parameter $C \in [0.001, 1000]$ was tuned on the linear kernel to establish a baseline. For the RBF kernel, first C was tuned while holding $\gamma = \text{'scale'}$, then γ values (0.001 to 10) were tuned with the best C . Since both the datasets had class imbalance, a balanced class-weighting scheme was used that inversely adjusts weights to the class frequencies.

For k-Nearest Neighbors, the model was evaluated across $k \in [1, 41]$. In the script `weights='distance'` is used to ensure neighbors are weighed by distance. This helps improve sensitivity to the minority classes in both datasets and better captures local structure in overlapping regions. For the Adult dataset, Manhattan distance was used because of high-dimensional variables. Due to the curse of dimensionality and sparse data, Manhattan distance is preferred as it measures absolute difference instead of computing squared differences. However on the Wine dataset, Euclidean distance is used as features are all continuous and the feature space is not affected by sparsity.

The Neural Network implementation involved two architectures: a "Wide" configuration (270 neurons in a single layer) and a "Deep" configuration (two layers: 150 neurons and 100 neurons, respectively). The batch size is 256. In both Scikit-learn and PyTorch, Stochastic Gradient Descent (SGD) was implemented with a fixed learning rate and L2 penalty. To prevent the high-capacity models from memorizing the tabular noise, an early stopping mechanism was used. The training was stopped once the validation loss did not improve after 10 "patience" epochs and the weights were reverted to the best-recorded state. The training and validation sets were then concatenated for a final evaluation. A 5-fold stratified cross-validation was used to ensure each fold maintained the original ratio of income classes. The hardware used for all experiments was CPU: x86_64, OS: Linux 6.6.105+. The experiments were run in Google Colab,

written in .ipynb notebooks.

Finally, a dummy classifier was trained on both datasets as a sanity check. This serves as a strict baseline model that always predicts the majority class. For the Wine quality dataset (8 classes), the dummy model predicts only the dominant class (class 4), resulting in 0.35 accuracy and an extremely low macro F1 of 0.064. This confirms that no leaky features (like duplicated class column) were used and that the train/test split was performed prior to modeling. Similarly, for the Adult dataset, the dummy predicts only the majority class ($\leq 50K$), achieving 0.75 accuracy but a macro F1 of 0.43 and zero recall for the $>50K$ class. This high accuracy is due to the class imbalance.

All trained models substantially outperform these baselines, particularly in macro F1 and minority-class recall. The consistent improvement over the dummy baseline provides strong evidence that leakage was successfully controlled. Although not all present in this report, results for AUC-ROC, accuracy, recall, and precision for all of the models can be obtained in AdultModels.ipynb and WineModels.ipynb.

IV. METRIC COMPARISON

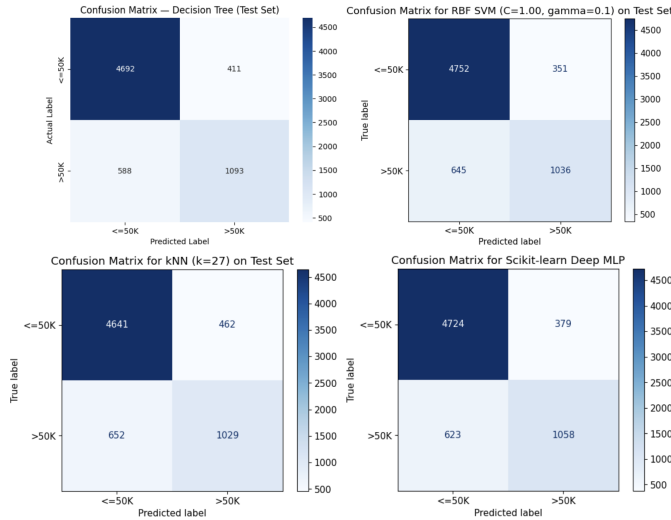


Fig. 3: Confusion matrices for all four models on the Adult dataset.

The results of the four models on the Adult Income dataset show a very similar accuracy on all four models in the range 0.84-0.85. The main differences come from the macro F1 and the recall of the $>50k$ class. This is expected as there was a high likelihood of Type II errors in this class due to the imbalance which encourages models to increase accuracy by defaulting to predicting $\leq 50k$. The recall of the $>50k$ class is consistently low across the models. The hypothesis is partially supported by the results Decision Tree (0.795) $\geq$ Neural Network (0.791) $\geq$ SVM (0.790) $\geq$ KNN (0.771). Neural networks perform slightly better than SVMs unlike the prediction. This is because SVMs are conservative with their classification of the $>50k$ points while neural networks have a higher recall with these.

Decision trees performed best with the highest macro F1 of 0.795, the best $>50k$ recall of 0.650, and good precision of 0.727. There are 411 false positives and 588 false negatives. These results reflect the properties highlighted in the hypothesis. The tree separates the majority class early using a greedy splitting strategy. Once a path isolates a $>50k$ subgroup, the model commits to it. DTs are better at learning specific rule combinations by making the best local splits that give highest information gain. The depth (24) and leaves (211) of this DT are much lower than those observed on the Wine tree which shows good generalization of rules after pruning as well. SVMs have a higher precision of 0.745 on the $>50k$ class but the lowest recall of 0.597. They have the lowest false positives (351) and highest false negatives (645). This leads to a macro F1 of 0.783 which lower than DT and NN. SVMs sacrifice recall on the $>50k$ class to have a clean margin. Although SVMs were able to correctly separate the majority class in the high-dimensional space as predicted, they performed worse than NN due to low recall on the minority class.

Neural Networks have a slightly lower macro F1 (0.790) and $>50k$ recall (0.639) than DT but better than the rest of the models. The hypothesis predicted that Neural Networks might struggle with the noise, variance, and class imbalance in the dataset. However, the results show that after regularization and training, the NN generalized better than expected. This is because NN has a high representational capacity and can learn non-linear interactions that can separate the classes better. Neural networks have more false positives than SVM (379) and less false negatives (623), making them more effective at identifying $>50k$ class overall. kNN has the lowest macro F1 (0.771) and weak $>50k$ precision and recall as predicted. They have the highest false positives (462) and the highest false negatives (652). Since kNNs are highly influenced by neighbors, the domination of $\leq 50k$ data points causes the $>50k$ points to be misclassified as expected.

There is a wider range of accuracies across the four models on the Wine Quality dataset (0.56-0.63) and a similar range of macro F1 (0.42-0.45). The results do not align with the hypothesis on the wine dataset. The observed ranking was kNN (0.4359) $\geq$ Decision Tree (0.4299) $\geq$ SVM (0.4224) $\geq$ Neural Network (0.3662). Neural networks perform worst unlike predicted and kNN substantially outperforms expectations and surpasses all the models. The SVM also performed worse than expected.

One of the biggest deviations from the hypothesis is kNN, which was expected to perform worst but instead achieves the highest macro F1 (0.4359) and the highest weighted F1 (0.6804). kNN has poor performance on minority labels as predicted. The kNN model has the highest Type II errors among the models and occasionally produces no true positives at all. This is due to the curse of imbalance and the model's reliance on local neighborhoods, which are dominated by the majority classes. For majority classes such as 4 and 5, kNN makes fewer Type II errors and benefits from smoothing due to the large neighborhood size ($k = 39$).

Neural Networks have the lowest macro F1 (0.3662).

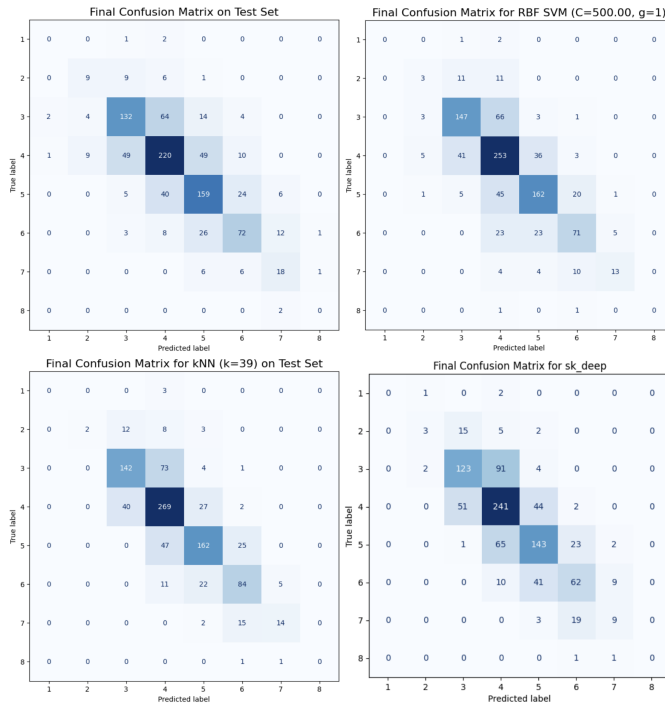


Fig. 4: Confusion matrices for all four models on the Wine dataset.

This is very unexpected due to the high learning power of NNs. Neural networks require substantial training data to generalize effectively, and in small tabular datasets they often overfit and bias toward dominant patterns (Goodfellow et al., 2016). The size of the dataset is limited (6497) and the limited samples for extreme wine qualities (classes 1, 2, 7, and 8) lead the model to "play it safe" by over-predicting the majority middle classes (4, 5, and 6) and failing to generalize the rarer patterns. The model has the lowest number of true positives for the rarer classes showing that it has high variance and clearly overfits the training data.

The results on decision tree matches the hypothesis that it shows high variance on the noisy, continuous data. It has a lower macro F1 of 0.4299. Even on majority classes there are substantial Type I and Type II errors. This reflects axis-aligned splits causing unstable local decisions due to the non-linear feature space. Minority classes suffer heavily, the tree has no true positives for some of the rarest labels and produces only false negatives. Although pruning was applied, the final depth (31) and the large number of leaves (1601) lead to unstable boundaries that do not generalize well to minority classes.

The RBF SVM performs the second worst, not aligning to the hypothesis. It has a macro F1 of 0.4224. There is low Type I errors but consistently high Type II errors. The SVM seems to be more affected by the imbalanced dataset and margin maximization than expected. The model plays it safe with the decision boundary, assigning uncertain samples to majority classes rather than risking false positives. This leads to a low recall even in majority classes. For rare classes

(1,8), the SVM collapses completely, producing only false negatives and no true positives. It has fewer true positives in (1,2,7,8) than all the other models. Therefore, the SVM reduces overall false positives in majority classes (Type I errors) but increases false negatives in minority classes, leading to lower macro F1.

V. STABILITY AND HYPERPARAMETER SENSITIVITY

Across all models on both datasets, the corresponding hyperparameters were picked according to highest validation score after testing a range of values on the validation set. These are seen as the best points in the model-complexity curves provided and the validation/test scores are obtained using these hyperparameters.

The learning curves for Decision Tree in the Adult Dataset shows us the trade-off between instability (caused by variance) and bias. At lower training sizes, the training score is near perfect while cross-validation accuracy is lower indicating high variance. As the training size increases, the gap between the training and validation performance reduces. This shows that more data acts as a regularizer, the same is the case with pruning. The model-complexity curve shows us that lower values of `cpp_alpha` gives us the highest values of F1 score while higher values leads to a sharp performance drop due to underfitting. Higher values of `cpp_alpha` lead to more pruning and a simpler model. With a simpler model, the trees lose specific patterns needed to predict minority classes, increasing type 2 errors. This is consistent with the learning curves. Overall, Decision Trees are unstable as shown by the sharp drop in F1-score with a small increase in `cpp_alpha` and the initial high variance in learning curves. However, choosing the correct `cpp_alpha` helps reduce this instability and increasing the training data size increases the model's generalization.

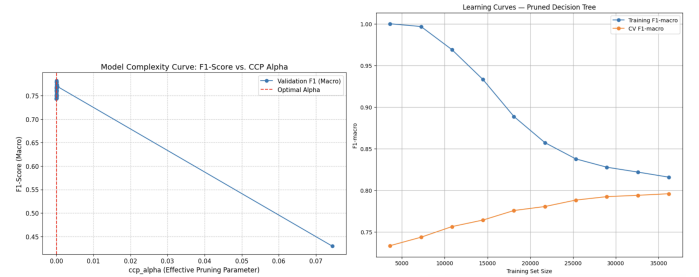


Fig. 5: Model complexity and learning curves for the decision tree model on the Adult dataset.

On the Wine dataset, Decision Trees show very high capacity but low stability through the complexity and learning curves. The model has perfect training F1 (around 1.0) across all training sizes, while cross-validation F1 remains much lower (0.40–0.43), indicating strong overfitting. This shows that the model is complex enough to learn all the patterns in the training data however it is not able to generalize like it does on the adult dataset. The Wine dataset `cpp_alpha` complexity curve also has a steep drop in validation F1. Any pruning results in lower performance, hence generalization

cannot be improved through this. Overall, the model is just not able to perform well on this dataset as axis-aligned splits are not able to capture the very complex non-linear patterns in this dataset resulting in the low F1 score.

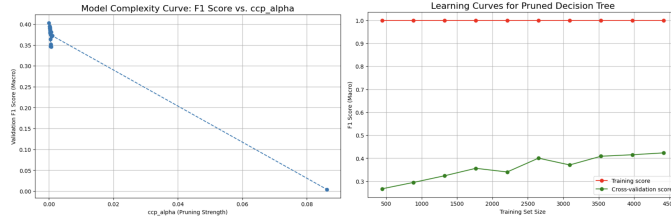


Fig. 6: Model complexity and learning curves for the decision tree model on the Wine dataset.

The model-complexity and learning curves on the Adult dataset show us that SVMs have strong stability once tuned. Both a linear and RBF kernel were trained on the data. The linear SVM showed a flat validation score across many increasing C values. This shows good stability. The linear SVM performs well with a final score of 0.7663. The tuning process for the RBF kernel was different than the linear kernel as the RBF kernel needs two parameters, C and γ , to be tuned. C was initially tuned with a default γ and the best C was used to tune γ after. The sensitivity of the model's hyperparameters was found to be greater in the RBF kernel which is evidence that it has a greater capacity than the linear kernel. The C parameter is not as sensitive as γ but has higher F1-scores than (0.745-0.770) and shows a drop in score with increase in C beyond $C = 1$. There is a bigger range of scores (0.445-0.755) with γ in the RBF kernel with a steep drop after $\gamma=0.1$, indicating high sensitivity. Overall, the RBF kernel performed slightly better with a score of 0.7698. The learning curves for the RBF kernel also show that the model has a good capacity and captures the non-linear patterns in the data points. The model's training score is high which indicates that it is able to learn a lot of the training data. The gap between training and cross-validation F1-score of the RBF SVM is smaller than that of DTs, showing greater stability. While the RBF SVM achieves high overall accuracy and strong precision for the $>50k$ class through its smoother decision boundary, its recall remains limited as points from the $>50k$ class are also misclassified due to this.

SVMs have lower capacity than trees on the Wine Dataset. The linear SVM complexity curve shows the limitations of the model as the peak macro-F1 is very low around 0.33. The RBF Kernel has a higher capacity than the linear kernel with the highest F1 macro being 0.4143. There is some hyperparameter sensitivity but not as much as DTs. In the model-complexity curve for C , the scores are in a range of 0.32-0.40. Similarly, all the scores in the γ graph are in small range of 0.32-0.42. The scores are overall higher than those in the linear C graph, showing higher capacity. There is moderate hyperparameter sensitivity as seen in the score changes with change in hyperparameter magnitude. The RBF kernel was therefore picked as the better model

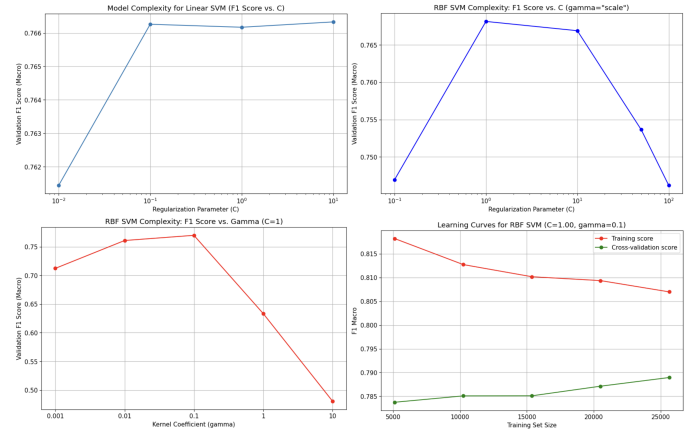


Fig. 7: Model complexity curves for linear and RBF kernels, and a learning curve for RBF kernel for the SVM model on the Adult dataset.

and tested on the validation set. The learning curves show a large gap between the training and validation curves, similar to DTs. The training scores are perfect, like DT indicating that the model has a high capacity and poor generalization. The gap narrows slowly with more data. However, the highest validation F1 score is still pretty low, close to the decision tree's highest score. Overall, the learning curves are nearly identical to decision trees. Both models show high capacity but poor generalization on this dataset. Similar to the Adult dataset, the model suffers from misclassifications on the minority classes due to the smooth decision boundary.

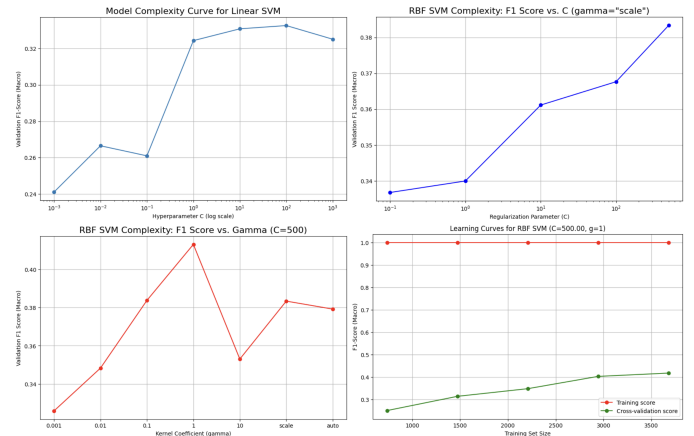


Fig. 8: Model complexity curves for linear and RBF kernels, and a learning curve for RBF kernel for the SVM model on the Wine dataset.

The model-complexity and learning curves for kNN on the Adult Dataset show that it is sensitive to k and has limited capacity on the Adult dataset. The validation F1 score curve shows a big improvement as k increases from very small values as there is reduction of variance as predictions become less dependent on noisy local neighbors. As k increases, the F1-score plateaus around 0.83. However, further increases in k have either small increases or decreases

in F1-score. This is because the predictions become more biased towards the majority $\leq 50K$ class. This causes lower minority ($>50K$) recall, as the $\leq 50k$ points dominate the space. The learning curves for kNN at the selected k value reinforce this. Training and cross-validation F1-scores are relatively close across training sizes, suggesting that the model does not do a good job picking up the intricate patterns needed to separate the classes in either case initially. Both curves improve gradually as more data is added but plateau at a lower F1-score compared to DTs and SVMs. This is because of kNN's reliance on distance-based similarity of points, which is less informative in this setting due to the curse-of-dimensionality. kNN shows low stability and poor performance compared to models such as neural networks or SVMs. Overall, these curves confirm that while increasing k stabilizes kNN predictions, it also increases $\leq 50K$ class bias, making kNN the most error-prone model in this dataset.

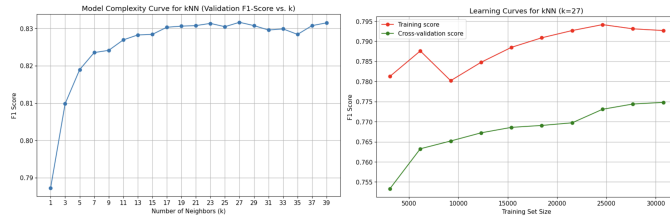


Fig. 9: Model complexity and learning curves for the KNN model on the Adult dataset.

The stability of the kNN model on the wine quality dataset is also sensitive to the choice of k. The model complexity curve shows us that the F1-score fluctuates with different values of k, showing instability. The optimal validation F1-score is slightly above 0.45 and has a very high k of 39. This means that a larger neighborhood of data points is needed to make stable predictions, smoothing out the decision boundary. Since the rest of the models perform so poorly on the dataset, kNNs score is still relatively higher, this is mostly achieved due to blind averaging over a high number of neighbors.

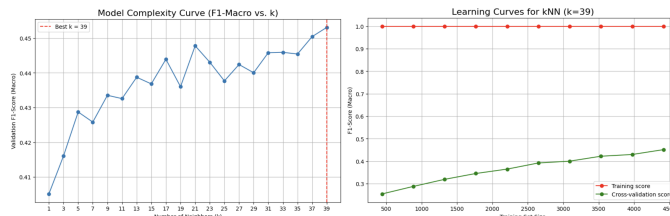


Fig. 10: Model complexity and learning curves for the KNN model on the Wine dataset.

Both pytorch and scikit-learn Neural Network models were trained on the Adult Income dataset. Out of all four models, the deep architecture performed better in both pytorch and sklearn and the pytorch deep MLP (Multi-layer Perceptron) performed best overall. The minimum loss across all models was very close (0.3213-0.3229). However, the

deep model's hierarchy allows it to learn the non-linear interactions between the features better which improves its performance. The learning curves show that the Deep MLP is a stable model. The training and cross-validation curves have increasing F1-scores which plateau with a small gap between them. This is an indicator of stability and shows low variance due to good generalization. Overall, these observations reflect that high-capacity models without strong regularization (Decision Trees, kNN) are more unstable, while optimization-based models (SVMs and neural networks) have greater stability but require careful hyperparameter tuning to balance bias, variance, and recall of the $\leq 50k$ class.

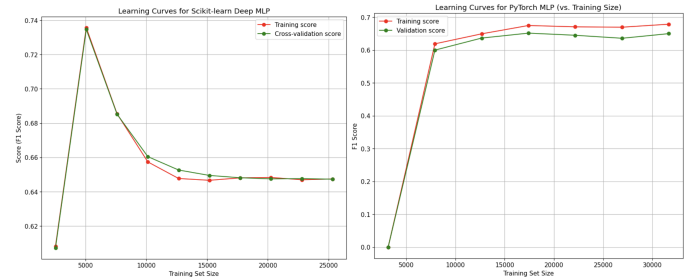


Fig. 11: Learning curves for neural network models (Scikit-learn and Pytorch MLP's) on the Adult dataset.

The same training process was used for Neural Networks on the Wine Quality dataset. The Sklearn deep model performed best with the lowest loss here. The losses were pretty close between 1.02-1.08, but both the sklearn models performed better than the pytorch models. The learning curves for sklearn deep model has a gap between training and validation scores that narrows slightly with more data. The gap in F1-score is smaller than that of the other models (< 0.14). This is because the training F1-score is much lower in this model than other models indicating that it struggles to learn general patterns. The final validation score is very low, around 0.37. However, this is still better than other NN models on the test set. The learning curves for the pytorch model show a smaller gap between validation and training scores but a lower overall score (around 0.34) indicating low capacity and performance. The model is not as stable on the Wine dataset as on the Adult dataset as there are larger gaps in learning curves indicating more variance. The learning curves on both models show that that NNs struggle to learn good predictive patterns in the data and perform poorly in this setting.

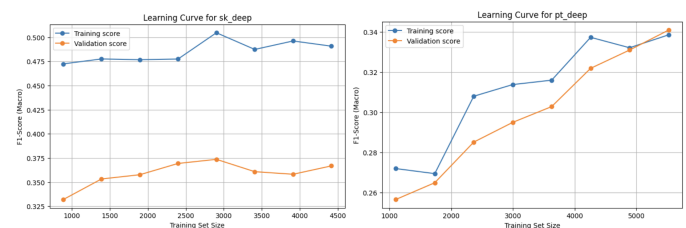


Fig. 12: Learning curves for neural network models (Scikit-learn and Pytorch MLP's) on the Wine dataset.

VI. COMPARISON OF NN MODELS

On the Adult Income dataset there were a few differences noticed in the training process of the two models. Comparing the learning curves of the two models on the deep architecture (the winning architecture with lowest loss on both models), we can see that sklearn's training and validation curves hit a peak early (around 0.74 F1 score at 5000 samples) and drop afterwards to close to 0.645. This is because sklearn's NN implementation isn't designed to scale with large datasets. Therefore, as the dataset grows, the model doesn't get optimized enough to make good predictions. The pytorch learning curves keep getting better scores as dataset size increases, increasing to around 0.68 F1 score before plateauing. This shows that the pytorch model has more capacity and keeps growing and generalizing with more data unlike sklearn that hits a limit early on. However, both models perform similarly on the test set with sklearn doing slightly better (0.7914 vs 0.7901)

The epoch-loss curves, compared across the four models are pretty similar. The models with the deep architecture have a slightly lower loss than the wide architecture. This is because the complexity of the deep models help capture more intricate non-linear patterns between the features. Overall, the pytorch deep model has the lowest loss of 0.3214. However, sklearn is selected to compare across the rest of the models in this paper as it has a better F1-score. sklearn completed training in around 0.65 seconds, while PyTorch required around 59 seconds, despite similar architectures and epoch counts. This is because scikit-learn runs highly optimized Cython loops but PyTorch executes many user controlled Python iterations per batch. However, prediction speed is very small for both the two frameworks (0.010-0.025) as it only requires a few matrix multiplications once model training is done. In terms of runtime, pytorch is much slower and produces worse results, hence sklearn is the better model here.

On the Wine Quality dataset, sklearn performed very differently. The training score on sklearn deep model is consistently much higher than on the pytorch model showing that the model has a higher capacity. However the curves show that sklearn overfits and pytorch generalizes better from the smaller gap. The highest training score is 0.34 on pytorch but 0.50 on sklearn. This shows that on this particular dataset blind overfitting on the intricate patterns is more beneficial than the generalization provided by pytorch. Ultimately Sklearn has a higher F1-score of 0.3662 than Pytorch with 0.3283 on the test set.

The epoch loss curves are also pretty similar on the wine dataset. The most apparent difference being the gap in both the sklearn models compared to close to no gap at all in the pytorch models. This is consistent with the learning curves where sklearn overfits more than pytorch. Sklearn performs much better on this dataset because the Wine Quality dataset is small and has lower dimensions. Hence, it can memorize the patterns within this dataset better. The L2-regularization parameter was chosen to be 0.01 after testing across a few

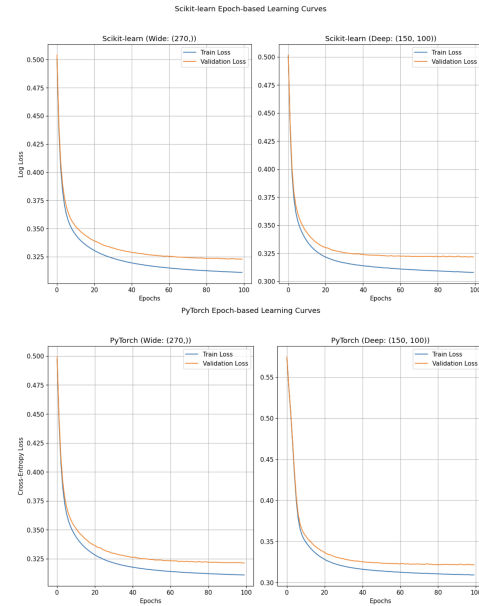


Fig. 13: Loss curves for neural network models (Scikit-learn and Pytorch MLP's) on the Adult dataset.

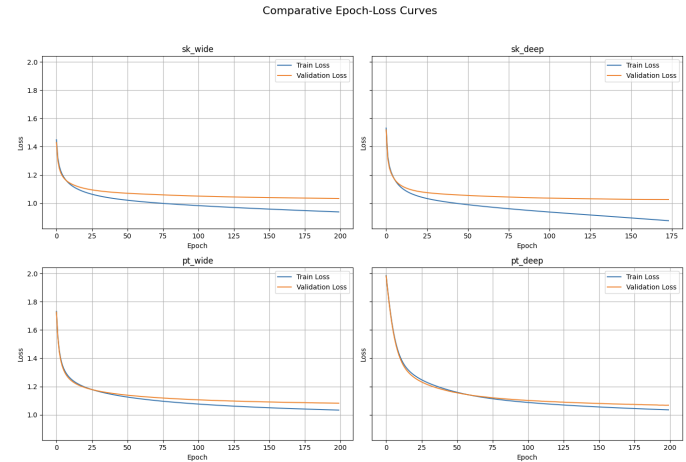


Fig. 14: Loss curves for neural network models (Scikit-learn and Pytorch MLP's) on the Wine dataset.

values on the pytorch deep model on both datasets as a sanity check.

Comparison of four activation functions (ReLU, GELU, SiLU, tanh) was performed on the Pytorch Deep model (Adult Income dataset). All functions have fast and stable convergence on the graph. Their loss curves drop quickly and smoothly which shows that learn efficiently with SGD. Tanh converges to a higher loss than the other functions. The main trade-off is between performance and inference speed. ReLU has the best performance (highest F1-score of 0.793, lowest loss) but has the second highest prediction speed (20.6s). The trade-off for using GELU or SiLU is that you sacrifice a tiny amount of accuracy for the fastest inference time. For Tanh, there is no positive trade-off; it is the slowest to train, the second-slowest at making predictions, and the least accurate,

making it the worst choice of the four.

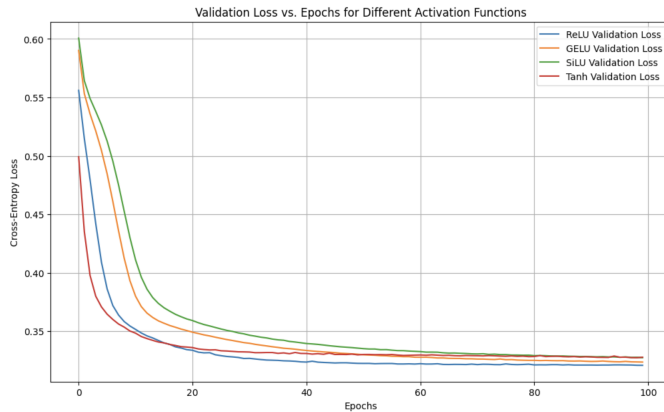


Fig. 15: Loss curves for four different activation functions on the Adult dataset.

VII. WALL CLOCK COMPARISON

K-Nearest Neighbors is almost instant to train on both datasets (0.1157s-A, 0.0141s-W), because it just stores the training set, but it's relatively slower at inference (2.4824s-A, 0.0813s-W) since every prediction searches the stored points. Decision Trees spend more time (1.6531s-A, 0.069s-W) to build splits, then predict very quickly by traversing the tree (0.0191s-A, 0.0034s-W). The SVM (RBF kernel) is the slowest on both ends, fitting is slow (126.05s-A, 2.4575s-W) due to the quadratic optimization over all pairs and predicting (14.89s-A, 0.3505s-W) is affected by scaling by the number of support vectors. The scikit-learn deep neural net is slower to train in the datasets (0.5699s-A, 11.3521s-W) due to many gradient-descent epochs but delivers extremely fast predictions (0.0268s-A, 0.0033s-W) compared to the training time. In short, KNN is cheapest to train but expensive to serve, trees are balanced with very fast inference, the neural net is training-heavy but inference-light, and the non-linear SVM is costly for both training and prediction.

VIII. CONCLUSION

The hypothesis was partially confirmed on the Adult dataset but largely did not align with the Wine dataset results. Decision Trees show very high capacity but low stability. They performed well on Adult but their instability was great on Wine, as its nonlinear relationships are harder to capture with axis-aligned splits. SVMs have low Type I error at the expense of high Type II error in both datasets. The model is willing to sacrifice recall of points in minority classes to improve accuracy. kNN consistently shows limited capacity and difficulty separating minority classes. On the Adult dataset, due to many encoded categorical variables, kNN performs poorly. This same is seen on the Wine dataset, although it performed better than expected due to high precision from averaging neighbors on the majority classes. Neural Networks performed nearly as expected on the Adult dataset but was impacted more than expected by the limited size and variance of the Wine dataset, performing the worst.

The overall performance was significantly different on each of the datasets. As predicted in the EDA, models perform poorly on the wine dataset due to the multi-class classification paired with the lack of collinearity between the quality feature and the rest of the features. Both accuracy and F1-scores were much lower on this dataset. There is little relation between chemical features and quality, and there are many random variations and measurement errors. Additionally, the wine quality feature is a subjective rating from a human taster. The relationships between the features are very complex and nonlinear. In the case of wine dataset, it would be more helpful to switch to training on Ensemble methods. Combining the predictions of multiple models like in Random forests can smooth out noisy and unstable decisions of individual trees. Additionally, Gradient Boosting models build trees sequentially, with each new tree correcting the errors of the previous ones. This is also ideal for this dataset. Another option would be to simplify the problem from mapping wines to eight classes to a smaller number of classes (for example, 2 or 3). This creates a clearer separation between classes which makes it easier to create a more generalizable decision boundary. Since there are so many features with low collinearity with the quality feature, it would also be useful to obtain the feature importance of each variable and using the top 5-6 features to predict the classes instead.

The Adult dataset had higher scores, partly due to the 3:1 class imbalance and simpler binary classification. The class imbalance allows the models to fall back on predicting the majority class for tricky data points and still be mostly correct. Future improvements on the Adult Income dataset should focus on improving detection of the >50K class rather than increasing overall accuracy. The performance also is currently limited by class imbalance. It would be useful to incorporate class-sensitive learning by applying class weights or cost-sensitive loss functions so that false negatives for the >50K class are penalized more heavily during training. Since Decision Trees performed best individually, it would be useful to extend them into ensemble methods such as Random Forests or Gradient Boosting here as well. This would likely improve stability and minority recall. Finally, rebalancing the training data through oversampling of >50K instances or undersampling of $\leq 50K$ cases could help models learn the minority class structure more effectively and reduce the Type II errors observed across all models.

AI Use Statement

ChatGPT and Google Gemini were used to generate templates for training and testing parts of the models. Additionally, after completing the report I used Google Gemini to correct spelling and grammar errors.

REFERENCES

- [1] H. He and E. Garcia, "Learning from imbalanced data," in IEEE Transactions on Knowledge and Data Engineering (TKDE), 2009, pp. 1263–1284. [Online]. Available: <https://ieeexplore.ieee.org/document/5128907>

- [2] I. Goodfellow, Y. Bengio, and A. Courville, "Deep Learning." MIT Press, 2017. [Online]. Available: <https://www.deeplearningbook.org/>